
What Went Wrong?

Structured Self-Debugging for LLM Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

When LLM agents fail at multi-step tasks, they produce execution traces containing every reasoning step, action, and observation that led to the failure. Unlike all prior software, these agents can re-read their own traces to diagnose and correct their mistakes, requiring no training, no external diagnosis, and no human intervention beyond a pass/fail signal. In this paper, we investigate whether and how this trace replay mechanism enables self-debugging. We introduce a structured method that forces the agent through systematic analysis at each retry — value estimation, root cause diagnosis, fix planning, and evidence accumulation — and compare it against unstructured replay and memoryless retry across coding and policy-following tasks. Our results show that self-debugging works: even unstructured replay solves a substantial fraction of tasks, and structured replay solves considerably more (37% of failed coding tasks, 71% of policy-following tasks). The benefit of structure compounds over iterations, the trace itself rather than the verifier drives the diagnostic signal, and partial progress after one round predicts whether further rounds will succeed. These findings suggest that the execution traces LLM agents already produce contain sufficient information for self-diagnosis, opening a path toward agents that improve continuously from their own mistakes.

1 Introduction

LLM agents automate workflows across software engineering, customer service, healthcare, financial trading, manufacturing, and many other industries [8]. Like all software, they fail. Unlike all prior software, however, they can read their own failure logs. These logs contain a complete execution trace of every reasoning step, action, and observation made by the agent. From this, the agent can generate a fix, retry, and in case of another failure – produce more trace. This unique debugging loop opens a question that could not be asked of any previous system: can LLMs fix their own mistakes?

If analyzing own failure logs and retrying works reliably, it could introduce a new form of self-improvement. An agent that can diagnose its own failures needs no external feedback beyond a pass/fail signal, no retraining, and no human intervention. It can improve continuously through deployment by replaying and correcting its mistakes. For this to work, the agent must do several things in sequence: localize which step caused the failure, diagnose why, generate a fix that addresses the root cause rather than a symptom, and verify that the fix worked. Each of these can go wrong. The agent may fixate on the wrong step [28], apply a superficial patch that does not address the underlying problem [16], or confirm its own flawed reasoning instead of genuinely verifying the fix [10]. As a result, the evidence on LLM self-correction is mixed. Studies of single-shot correction on reasoning tasks find it unreliable, with performance often degrading after correction [4, 6]. Agent systems that embed self-diagnosis as one component among many show it can work in practice [25, 29, 31]. The question of whether and how LLM agents can debug themselves from their own traces remains open.

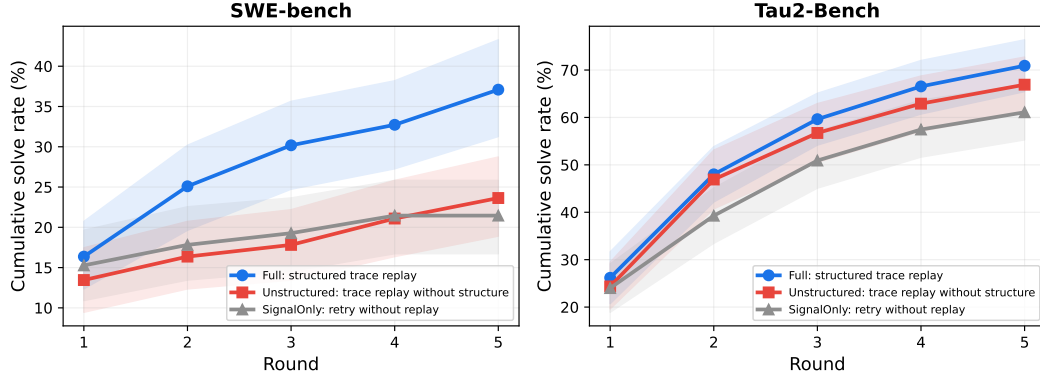


Figure 1: LLM agents can debug themselves by replaying their own execution traces. The strongest results are obtained when the agent is forced through structured analysis that localizes and fixes the most probable error cause. We show cumulative solve rates over five rounds on SWE-BENCH (code editing, left) and TAU2-BENCH (customer service, right), with 95% bootstrap confidence intervals ($n=275$, 1,000 resamples). As rounds progress, Full (structured trace replay) pulls ahead of Unstructured (trace replay without structure) and SignalOnly (retry without replay), with the gap widening as structured replay accumulates evidence across iterations.

In this paper, we investigate self-debugging in controlled experiments. We design a structured method that forces the agent to systematically analyze its trace before retrying to counter possible self-debugging failure modes, and compare it against unstructured replay and memoryless retry on two benchmarks that differ in domain, action space, and verifier richness (Table 1).

Our main contributions include:

1. We formalize self-debugging as a problem of iterative trace analysis, identifying four failure modes that unstructured approaches cannot address (Section 2)
2. We introduce a structured self-debugging method where each step targets a specific failure mode: value estimation to localize the error, root cause analysis to diagnose it, fix planning with predicted outcomes to verify it, and evidence accumulation to avoid repeating failed hypotheses.
3. We conduct a controlled empirical study across coding and policy following – two domains that differ in action space, trace structure, and verifier richness. Our results show that structured self-debugging solves 37% of previously failed coding tasks and 71% of policy-following tasks while being systematically better than unstructured approaches (see Fig. 1).

Related Work *Self-correction skepticism.* Our work is related to a stream of research that tested whether LLMs can correct their own mistakes and mostly found they cannot [see e.g., 4, 6, 16, 28]. Huang et al. [4] showed that models cannot reliably find errors through introspection alone. Olausson et al. [16] showed that self-repair is marginal when compute is properly controlled. Kamoi et al. [6] showed that positive results from Self-Refine [12] are explained by deliberately weak baselines. More recently, Miao et al. [13] found that Reflexion can cause significant performance drops on reasoning benchmarks. We build on this skepticism but study a different regime: iterative structured debugging with evidence accumulation, rather than single-shot self-correction.

Agent systems with reflection. Our work is motivated by agent systems that embed self-diagnosis as a load-bearing primitive. Reflexion [25] uses verbal reflection to improve retry. LATS [31] combines reflection with tree search. Voyager [29] uses a critic agent for task verification. These systems demonstrate that some form of self-analysis can work in practice, but none isolate *whether the structure of reflection matters* versus just retrying with more context. Our ablation study directly tests this by comparing structured self-debugging against unstructured alternatives at matched compute.

Intervention-driven debugging. Our method is adapted from DoVer [11], which introduces intervention-driven debugging for multi-agent systems. DoVer segments a failed execution log, hypothesizes which agent failed, generates a targeted intervention, and replays from a checkpoint. We adapt this framework to *self-debugging*—where the model analyzes its own trace rather than an external system’s—and extend it with iteration and evidence accumulation, which DoVer does not support.

71 *Training-based self-correction.* A parallel stream trains models to self-correct. SCoRe [7] uses RL
 72 to make single-model self-correction work (+15.6% on MATH). Agent-R [30] identifies the first
 73 error step in failed trajectories and uses MCTS to construct corrective recovery paths. SETS [22]
 74 shows that when verification quality is low, self-correction provides minimal benefit—self-correction
 75 amplifies capability but does not create it. These methods require training; our method works at
 76 inference time with any model.

77 *Credit assignment and experience replay.* Our method performs credit assignment over execution
 78 traces, identifying which step caused the failure. This is the same fundamental problem studied
 79 in reinforcement learning [26, 27]. Our evidence accumulation mirrors experience replay [9, 14],
 80 and our root cause analysis—specifying what should have happened—echoes hindsight experience
 81 replay [1]. The key difference is that our method operates entirely through the context window at
 82 inference time, without parameter updates. In this sense, our work studies whether LLMs can perform
 83 the RL credit assignment loop *in context* rather than through gradient-based learning.

84 2 Problem Statement

85 We consider a setting where an LLM agent M executes a multi-step task $\mathcal{T}$, producing a trace
 86 $\tau = (q_1, q_2, \dots, q_T)$ where each q_t is one of:

- 87 • $q_t^{\text{reasoning}}$: a reasoning step (the model’s internal chain of thought),
- 88 • q_t^{action} : an action step (a tool call, file edit, API request), or
- 89 • $q_t^{\text{observation}}$: an observation step (the environment’s response to an action).

90 A verifier $V(\tau) \in \{\text{pass}, \text{fail}\}$ determines whether the task was completed successfully. We focus on
 91 the case where $V(\tau) = \text{fail}$: the agent has failed and was given a natural language signal / complaint
 92 Q indicating that something went wrong.

93 Our goal is to determine whether the agent can use its own trace τ to identify and correct the mistake,
 94 without external diagnosis.

95 2.1 Causal Structure

96 The trace τ has a natural causal structure: each step q_t depends on all previous steps $q_1, \dots, q_{t-1}$ and
 97 the task $\mathcal{T}$. The verifier outcome depends on the full trace:

$$V(\tau) = V(q_1, q_2, \dots, q_T). \quad (1)$$

98 When $V(\tau) = \text{fail}$, self-debugging asks *which step caused the failure?* We formalize this using the
 99 do-operator [24]. The *causal effect* of step t on the outcome is the change in the verifier result when
 100 we intervene on step t , replacing it with an alternative q_t^* :

$$\Delta_t = V(\tau \mid \text{do}(q_t = q_t^*)) - V(\tau). \quad (2)$$

101 A step q_t is a *root cause* of the failure if $\Delta_t > 0$, i.e., intervening on q_t alone is sufficient to change
 102 the outcome from fail to pass. Note that interventions propagate: $\text{do}(q_t = q_t^*)$ changes not just step t
 103 but all downstream steps $q_{t+1}, \dots, q_T$ that depend on it.

104 In practice, we cannot enumerate all possible alternatives q_t^* . Instead, the model generates a hypothesis
 105 about what q_t^* should be and tests it by re-executing. The self-debugging loop is a sequence of such
 106 causal interventions.

107 2.2 Failure Modes of Unstructured Self-Correction

108 When an agent retries a failed task without structure, it may fail to improve for several reasons. We
 109 identify four failure modes, each documented in the literature and each occurring at a different stage
 110 of the self-debugging process.

111 **Remark 1** (Misdiagnosis). *The agent identifies the wrong root cause. Models tend to fixate on the*
 112 *most salient error; typically the step closest to the failure, while missing the actual cause earlier in*
 113 *the trace [18, 28]. A wrong action at step 3 may corrupt all subsequent observations, yet the agent*
 114 *patches the symptom at step 10 while the cause persists. Error localization accuracy on reasoning*
 115 *chains remains around 53% [28], and models exhibit a 64.5% blind spot rate when correcting their*
 116 *own errors versus identical external ones [19].*

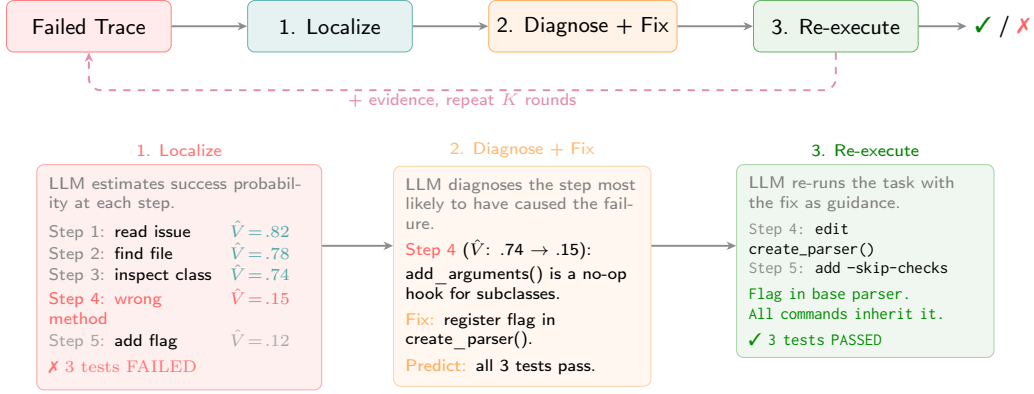


Figure 2: Overview of the structured self-debugging pipeline. We show the abstract method (top) and a concrete instantiation on SWE-BENCH task `django-11292` (bottom). The LLM estimates a success probability $\hat{V}$ at each trace step (left), diagnoses the step with the largest drop as the root cause (center), and re-executes with a targeted fix (right). The dashed arrow indicates iteration with accumulated evidence across K rounds.

Remark 2 (Misordering). *When multiple errors exist, the agent may fix an inconsequential one while the critical one goes unaddressed. Not all errors are equally fixable. Syntactic errors are repaired at substantially higher rates than semantic errors [21]. Spending an attempt on a hard, low-impact error when an easy, high-impact one exists wastes the opportunity.*

Remark 3 (Blind Verification). *After applying a fix, the agent lacks a mechanism to detect that its diagnosis was wrong. Wrong diagnoses actively mislead subsequent correction attempts [20], and 85–95% of self-verification steps merely confirm prior reasoning rather than genuinely checking the outcome [10].*

Remark 4 (Memory Failure). *Without structured memory, the agent either forgets what it tried or drowns in accumulated context. Self-correction can cause correct-to-incorrect regression, with accuracy dropping from 75.8% to 38.1% after a single round on CommonSenseQA [4]. Frontier models degrade with increasing context length [3], and iterative self-refinement can collapse into self-deterioration with 10–20% performance drops [2].*

In Section 3, we present a method that addresses each of these failure modes: value estimation and causal chains prevent misdiagnosis (Remark 1), prioritization by estimated impact prevents misordering (Remark 2), prediction verification detects blind verification (Remark 3), and structured evidence accumulation prevents memory failure (Remark 4).

We note that our method is deliberately designed as the strongest possible structured approach that addresses every documented failure mode. In practice, a deployed system would benefit from adapting the level of structure to the task at hand. We adopt the strongest approach because our primary goal is to determine whether structure helps at all, not to optimize for efficiency.

We also note that the verifier V plays a critical role in self-debugging. A rich verifier (e.g., a test suite that reports which tests pass and fail with error messages) provides detailed signal for diagnosis. A thin verifier (e.g., binary correct/incorrect) provides almost none. We study domains spanning this spectrum in Section 4.

3 Method

We present our structured self-debugging method in Algorithm 1 and illustrate it with a concrete example in Fig. 2. The method iterates for up to K rounds. Each round performs five operations that address the failure modes from Section 2.2.

3.1 Step-by-Step Description

Step 1: Value estimation and causal attribution (Remark 1). For each step t , the model estimates $\hat{V}(q_t) = P(V(\tau) = \text{pass} \mid q_1, \dots, q_t)$ —the probability of task success given the trace up to step t .

Algorithm 1 Structured Self-Debugging

Require: $\mathcal{T}, \tau_0, Q, M, V, K$

```
1:  $E_0 \leftarrow \emptyset$ 
2: for  $k = 1, \dots, K$  do
3:   for  $t = 1, \dots, |\tau_{k-1}|$  do Estimate value at each step
4:      $\hat{V}(q_t) \leftarrow M(\tau_{k-1}, Q, E_{k-1}, t)$ 
5:      $\hat{t} \leftarrow \arg \min_t \hat{V}(q_{t+1}) - \hat{V}(q_t)$  Find largest value drop
6:      $h_k \leftarrow \text{CAUSALCHAIN}(M, \tau_{k-1}, \hat{t}, E_{k-1})$  Trace cause to failure
7:      $q_t^*, \hat{O}_k \leftarrow \text{FIXPLAN}(M, \tau_{k-1}, h_k, E_{k-1})$  Generate fix + predict outcome
8:      $\tau_k \leftarrow \text{EXECUTE}(M, \mathcal{T}, (\hat{t}, q_t^*), E_{k-1})$  Re-execute with fix
9:     if  $V(\tau_k) = \text{pass}$  then return  $\tau_k$ 
10:     $\delta_k \leftarrow \text{COMPARE}(M, \hat{O}_k, V(\tau_k))$  Compare predicted vs. actual
11:     $E_k \leftarrow E_{k-1} \cup \{(h_k, q_t^*, \hat{O}_k, V(\tau_k), \delta_k)\}$ 
```

Output Best trace from $\{\tau_1, \dots, \tau_K\}$

149 The step with the largest negative change $\hat{t} = \arg \min_t \Delta V_t$ is the root cause candidate. The model
150 then produces a structured hypothesis:

$$h_k = (\hat{t}, \text{error at } q_{\hat{t}}, \text{ what should have happened, why, } q_{\hat{t}} \rightarrow q_{t'} \rightarrow \dots \rightarrow V(\tau) = \text{fail}). \quad (3)$$

151 We implement this as a structured prompt that requires the model to fill in each field: the root step,
152 what went wrong, the counterfactual alternative, the mechanism, and the propagation chain. Value
153 estimation forces the model to examine every step rather than fixating on the most salient error. The
154 causal chain constrains interventions to the root cause rather than downstream symptoms. Together,
155 performing this analysis is aimed to alleviate misdiagnosis (Remark 1).

156 **Step 2: Counterfactual fix plan (Remark 2).** Given the diagnosed root cause, the model proposes
157 an alternative action q_t^* for the identified step. Along with this fix, the model predicts what should
158 change in the verifier output if the fix is correct. For example, it may predict that a specific test should
159 now pass, or that a particular error message should disappear:

$$\hat{O}_k = \{\text{“check A should pass”, “check B should remain unchanged”, } \dots\}. \quad (4)$$

160 These predicted outcomes serve two purposes. First, they force the model to commit to a concrete,
161 falsifiable hypothesis about why the fix should work. Second, they provide a basis for verification in
162 Step 4. When the model identifies multiple errors, it prioritizes by $|\Delta V_t|$, fixing the step with the
163 largest value drop first. This prevents the misordering failure mode (Remark 2), where the agent
164 wastes an attempt on a low-impact error.

165 **Step 3: Re-execution.** The task is re-executed with the fix plan as guidance, producing trace
166 $\tau_k = M(\mathcal{T}, I_k, E_{k-1})$.

167 **Step 4: Prediction verification (Remark 3).** After re-execution, the model compares each predic-
168 tion in $\hat{O}_k$ against the actual verifier output $V(\tau_k)$ and produces a structured prediction error:

$$\delta_k = (\text{correct predictions, wrong predictions, unexpected outcomes, revised diagnosis}). \quad (5)$$

169 For example, if the model predicted “check A should pass” but check A still fails, the prediction
170 was wrong—and if check C now fails unexpectedly, the fix introduced a side effect. In both cases,
171 the mismatch reveals that the causal chain in h_k was incorrect. The model uses these mismatches
172 to revise its diagnosis before the next round. This directly addresses blind verification (Remark 3):
173 rather than observing only pass/fail, the model evaluates *why* the fix did or did not work.

174 **Step 5: Evidence accumulation (Remark 4).** The model appends the round’s complete record to
175 a growing evidence set:

$$E_k = E_{k-1} \cup \{(h_k, q_t^*, \hat{O}_k, V(\tau_k), \delta_k)\}. \quad (6)$$

176 Each entry captures what the model hypothesized, what fix it tried, what it predicted would happen,
177 what actually happened, and where the prediction was wrong. Because E_k grows monotonically, the
178 model cannot lose track of previously refuted hypotheses or silently repeat a failed fix. The structured
179 format keeps the signal-to-noise ratio high as the context window fills—addressing both aspects of
180 memory failure (Remark 4): the model retains what it learned, and can retrieve it efficiently.

3.2 Connection to Reinforcement Learning

Our method mirrors the reinforcement learning loop for credit assignment [see 27, Chapter 17], but operates entirely through the context window. In TD learning [26], an agent maintains a value function $V(s)$, computes the TD error $\delta = r + \gamma V(s') - V(s)$ after each action, and updates its policy accordingly. Our five steps implement the same cycle:

- *Value estimation.* Step 1 estimates $\hat{V}(q_t)$ at each trace step, analogous to $V(s)$. The largest drop identifies the root cause, just as the largest negative TD error localizes where things went wrong.
- *Credit assignment.* The causal chain in h_k traces how the value drop propagated from root cause to failure.
- *Policy improvement.* Step 2 generates an alternative action q_t^* with predicted outcome $\hat{O}_k$.
- *Prediction error.* Step 4 compares predicted against actual outcomes (δ_k), revealing whether the diagnosis was correct.
- *Experience replay.* Step 5 stores each round in E_k , analogous to experience replay [9, 14]. Specifying counterfactual alternatives echoes hindsight experience replay [1].

The key difference is that no weights are updated—the model “learns” by conditioning on the growing evidence set E_k in its context window.

4 Experiments

In this section, we present an empirical study on self-debugging in LLM agents. We include additional details in Section A, and code to reproduce our results on [our anonymized repository](#). We test the following hypotheses:

- H1. Structure matters beyond trace access.** Forcing the agent through systematic diagnosis solves tasks that unstructured replay and memoryless retry cannot.
- H2. Structure compounds over rounds.** The advantage of structured debugging widens across rounds as evidence accumulates, while unstructured methods plateau.
- H3. Richer verifiers help more.** Self-debugging accuracy increases with the richness of the verifier signal.

Setup Datasets. We select two benchmarks that differ along every dimension relevant to self-debugging—domain, action space, trace structure, and verifier richness (Table 1). If structured replay helps on both, the findings are unlikely to be specific to either benchmark.

- **SWE-BENCH [5]:** software engineering tasks where agents resolve GitHub issues by editing codebases. Traces come from SWE-agent with Llama-70B [15]. The verifier is a test suite that reports which tests pass and fail with detailed error messages and stack traces. This represents a *rich verifier* setting. We use 825 failed traces.
- **TAU2-BENCH [23]:** customer service tasks where agents follow complex policies—refund rules, fare classes, cancellation windows—by conversing with a simulated user and calling database tools. The verifier scores two dimensions independently: *communication* (did the agent say the right things?) and *database* (did the agent make the correct changes?), with a detailed diff explaining what was wrong. This represents a *structured verifier* setting—richer than binary pass/fail but organized around specific compliance dimensions rather than individual steps. We use 275 failed traces across 5 recent models (GPT-5, Claude Sonnet 4.5, Gemini 2.5 Pro, Kimi K2, Qwen3-Coder).

Methods. We compare three conditions that vary the level of structure imposed on self-debugging. All conditions use the same model, the same tasks, and the same compute budget of K rounds.

- **Full:** the structured method from Algorithm 1, executing all five steps per round.
- **Unstructured:** the model receives the full trace and failure signal but performs free-form analysis. It is asked to “find and fix the problem” without the five-step decomposition. This isolates whether the forced structure helps beyond trace access alone.

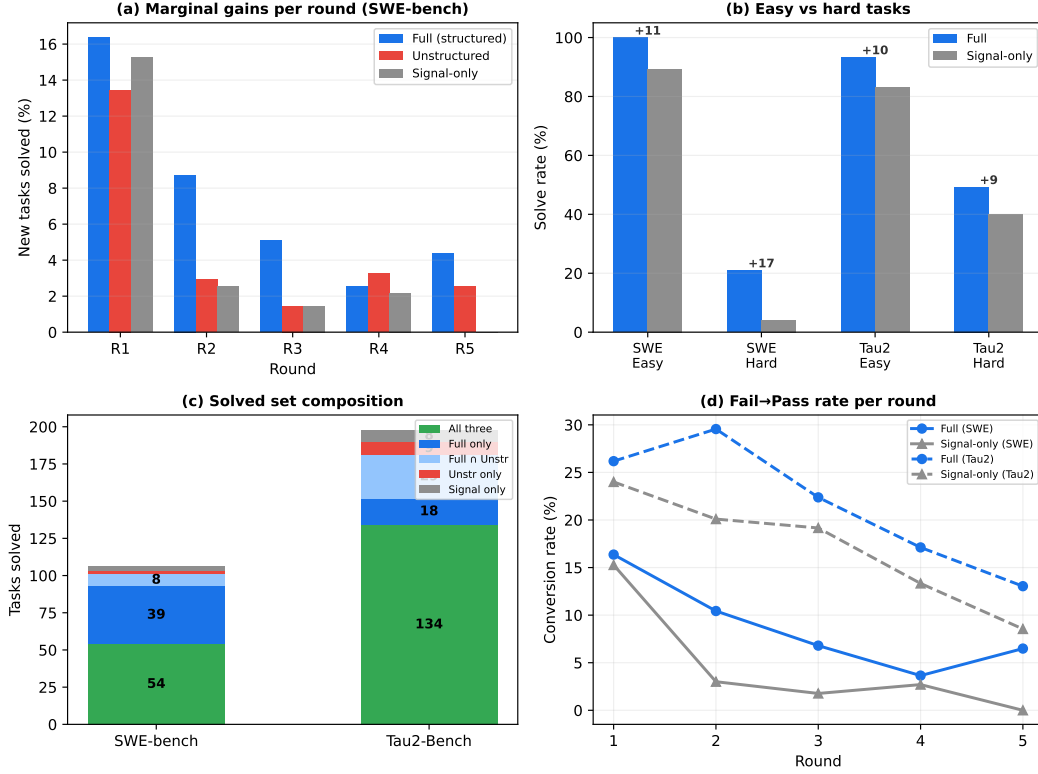


Figure 3: Analysis of self-debugging dynamics. We show (a) marginal gains per round on SWE-BENCH, where Full maintains higher new-solve rates across all rounds (H2); (b) solve rates stratified by task difficulty, where Full’s advantage is largest on hard tasks (H1); (c) the composition of solved task sets, where Full uniquely solves 39 SWE-BENCH tasks that no other condition reaches (H1); and (d) the per-round fail-to-pass conversion rate (H2).

222 • **SignalOnly:** the model receives only the verifier output (e.g., “test_foo FAILED”) and the original
 223 task, with no trace and no prior context. This is memoryless retry, which prior work has shown to
 224 be surprisingly competitive [16].

225 **Models.** We use GPT-5.4 [17], a frontier reason-
 226 ing model, as the debugger for all condi-
 227 tions. This provides a ceiling estimate of
 228 self-debugging performance. The debugger
 229 analyzes failed traces produced by other mod-
 230 els and re-executes each task with its proposed
 231 fix.

232 **Evaluation.** We evaluate each method using
 233 three metrics. All metrics are mechanical and
 234 require no LLM judges.

235 • **Success:** % tasks solved within K rounds.

236 • **Time to success:** the mean number of rounds to solve a task, among tasks eventually solved.

237 • **Diagnosis accuracy:** the fraction of rounds where the root cause targets the correct failure dimension.
 238 For TAU2-BENCH, we check whether the identified step type (tool call vs. message) matches the
 239 failing reward dimension (DB vs. COMMUNICATE). For SWE-BENCH, we check whether the
 240 diagnosis mentions the same file as the gold patch.

241 **Results** We present the main results in Table 2 and Figs. 1 and 3. In what follows, we discuss these
 242 results.

Table 1: Overview of datasets. We select two benchmarks that differ along every dimension relevant to self-debugging, so that consistent findings across both are unlikely to be benchmark-specific.

	SWE	TAU2
Domain	Code editing	Customer service
Actions	File edits, bash	NL + API calls
Trace	~10 steps	~40 steps
Verifier	Test suite (rich)	2 dims (crude)
Tasks	275	275

4.1 Structure Matters Beyond Trace Access (H1)

Full solves 37.1% of SWE-BENCH tasks and 70.9% of TAU2-BENCH tasks (Table 2). The Unstructured condition isolates the contribution of trace access: it receives the same trace and accumulates context across rounds, but performs free-form analysis instead of the five-step process. On SWE-BENCH, Unstructured gains only +2.1 points over SignalOnly, while Full gains +13.5 over Unstructured. On TAU2-BENCH, the split is more balanced (+5.8 from trace access, +4.0 from structure), suggesting that conversational traces are inherently informative even under free-form analysis. Full uniquely solves 39 SWE-BENCH tasks that neither ablation reaches (Fig. 3c). In total, these results show that simply giving the model its trace is insufficient; the forced five-step decomposition is what drives the improvement.

Table 2: We report solve rates (%) after $K=5$ rounds.

Condition	SWE	TAU2
Full	37.1	70.9
Unstructured	23.6	66.9
SignalOnly	21.5	61.1

4.2 Structure Compounds Over Rounds (H2)

On TAU2-BENCH, all three conditions solve a similar fraction of tasks at round 1 (24–26%). By round 2, Full reaches 48.0% while SignalOnly reaches 39.3%; by round 5, the gap has grown to 9.8 points (Fig. 1). On SWE-BENCH, Full leads from round 1 (16.4% vs. 13.5–15.3%) and maintains higher marginal gains at every subsequent round (Fig. 3a). Evidence accumulation and revised diagnoses give Full new information at each round, sustaining its per-round conversion rate (Fig. 3d). To illustrate, consider `django-11292`, which Full solves at round 4. The task requires adding a `-skip-checks` flag to Django’s management commands. The original agent placed the argument inside `BaseCommand.add_arguments()`, a no-op hook for subclasses. Our method diagnoses this as the root cause and specifies registering the flag in `create_parser()` alongside other global options. SignalOnly retries the same flawed approach because it lacks the trace-level diagnosis to redirect its strategy. These results indicate that the benefit of structured debugging is cumulative: each failed round produces evidence that narrows subsequent diagnoses, and this compounding effect is what separates Full from the ablations over multiple rounds.

4.3 Partial Progress Predicts Convergence

The fraction of verifier checks passed after one round predicts both eventual success and time to convergence (Table 3 and Fig. 4). On SWE-BENCH, tasks where the first attempt passes more than 75% of tests are solved 97% of the time in 1.0 rounds on average, while tasks with 0% passing are solved only 25% of the time and require 3.0 rounds. On TAU2-BENCH, tasks scoring above 0.5 at round 1 are solved 100% of the time. In practice, the initial pass rate serves as an early stopping criterion: a practitioner can decide after one round whether additional iterations are worth the compute.

Table 3: We report AUC and correlation (r) between initial pass rate and eventual success.

	AUC	r
SWE	0.74	−0.70
TAU2	0.71	−0.70

4.4 Contrary to H3, a Crude Verifier Suffices

On SWE-BENCH, where test failures include stack traces and error messages, Full improves by +15.6 points over SignalOnly. On TAU2-BENCH, where the verifier provides only two binary dimensions (database and communication correctness), the improvement is still +9.8 points. If verifier richness were the primary driver, the gap between benchmarks would be much larger. The structured replay mechanism extracts diagnostic signal from the trace itself, compensating for coarse verifier feedback. Within SWE-BENCH, success varies by repository: Full solves 57.8% of `sympy` tasks and 46.2% of `django` tasks, but 0% of `scikit-learn` and `astropy`. These repositories are hard for all conditions, indicating that the underlying bugs exceed model capability. Taken together, these results show that the trace itself, rather than the verifier, is the primary source of diagnostic signal, and structured replay is what unlocks it.

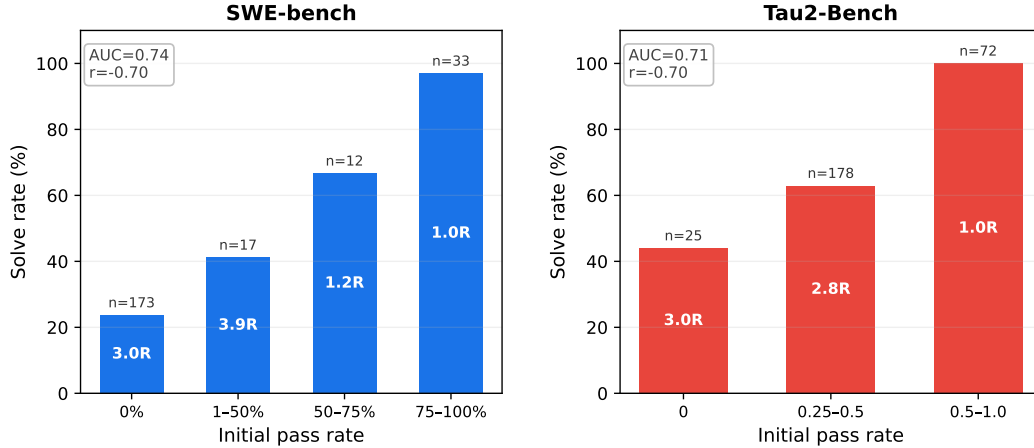


Figure 4: Solve rate stratified by initial pass rate on SWE-BENCH (left) and TAU2-BENCH (right). Numbers inside bars indicate mean rounds to solve. Higher partial scores at round 1 predict both higher solve rates and faster convergence.

5 Concluding Remarks

Summary of Contributions. We asked whether LLM agents can fix their own mistakes by replaying their execution traces. Our results show that they can. Even unstructured replay solves a substantial fraction of previously failed tasks on both benchmarks. Structured replay, which forces the agent through value estimation, root cause analysis, fix planning, and evidence accumulation, unlocks considerably more: 37.1% on SWE-BENCH and 70.9% on TAU2-BENCH. The benefit compounds over rounds, works even with crude verifier feedback, and is driven by the trace itself rather than the richness of the failure signal. These findings suggest that the execution traces LLM agents already produce contain sufficient information for self-diagnosis, and that the bottleneck is how systematically the agent analyzes them.

Limitations.

1. Our method requires the execution trace to be available in text form, limiting applicability to agents that log their reasoning and actions. Tasks with non-textual traces (e.g., robotic control) are outside our scope.
2. Our method assumes a verifier that provides at least a binary pass/fail signal. Tasks without automated verification require human feedback, reintroducing the dependence on external diagnosis that our method aims to remove.
3. We compare structured replay against unstructured replay and memoryless retry, but we do not ablate individual components of the method (value estimation, root cause analysis, fix planning, evidence accumulation) in isolation.
4. We evaluate with a single frontier debugger model (GPT-5.4). Whether the benefits of structured replay transfer to weaker models remains an open question.

Future Work. Our results establish that LLM agents can diagnose and fix their own failures from a single execution trace. This opens several directions. A natural next step is to develop *adaptive* self-debugging methods that adjust the level of structure based on task properties, applying full analysis when it helps and falling back to simpler retry when it does not. Our evidence accumulation mechanism currently resets between tasks; extending it to persist across tasks would enable agents to learn recurring failure patterns from deployment experience. More broadly, self-debugging is a first step toward autonomous self-improvement: an agent that can reliably identify why it failed and how to do better can, in principle, improve continuously without human intervention, retraining, or new data. Understanding the conditions under which this loop converges, and when it breaks down, is a central question for building reliable autonomous agents.

325
326
327
328
329
330
331
332
333
334
335
336
337

What Went Wrong? Structured Self-Debugging for LLM Agents

Supplementary Materials

A	Supporting Material for Section 4	11
A.1	Task Selection	11
A.2	Prompt Templates	11
A.3	Per-Round Solve Rates	12
A.4	Per-Repository Breakdown (SWE-BENCH)	12
B	Notation	12

338 A Supporting Material for Section 4

339 In what follows, we provide additional details on task selection, prompt templates, and per-round
340 results.

341 A.1 Task Selection

342 For SWE-BENCH, we use 275 failed traces from the Nebius SWE-agent trajectory dataset [15].
343 The original traces were generated by SWE-agent with GPT-4. We selected tasks from SWE-bench
344 Verified where the original agent submitted a patch that did not pass the test suite.

345 For TAU2-BENCH, we use 275 failed traces from five recent frontier models (GPT-5, Claude
346 Sonnet 4.5, Gemini 2.5 Pro, Kimi K2, Qwen3-Coder) on the airline domain. We selected tasks where
347 the original agent scored below the passing threshold.

348 All self-debugging rounds use GPT-5.4 [17] as the debugger model with a budget of $K = 5$ rounds.

349 A.2 Prompt Templates

350 We provide the system prompts for each step of the structured self-debugging method and for each
351 baseline condition.

352 **Step 1a (Value Estimation).** The model reads the execution trace and estimates the probability of
353 task success at each step.

354 You are reading an execution trace step by step and
355 estimating the probability of task success at each point.
356 After each step, estimate: given what has happened so far,
357 how likely is the task to succeed? Express as a probability
358 from 0.0 to 1.0.

359 **Step 1b (Root Cause Diagnosis).** Given the step with the largest value drop, the model diagnoses
360 what went wrong, what should have happened, why, and how the error propagated.

361 You are an expert debugger. The value estimates show
362 where the probability of task success dropped most sharply.
363 Analyze that step. Explain: (1) What happened, (2) What
364 should have happened instead, (3) Why the error occurred,
365 (4) How this error propagated through subsequent steps.

366 **Step 2 (Fix Plan).** The model generates an alternative action and predicts specific outcomes.

367 Given the root cause analysis, generate: (1) An alternative
368 action for the identified step, (2) Specific predicted
369 outcomes if this fix is applied. The predicted outcomes
370 must be specific and checkable against the verifier output.

371 **Step 4 (Prediction Verification).** The model compares predicted outcomes against actual results.

372 Compare predicted outcomes against actual results. For each
373 prediction, determine whether it matched. Then assess what
374 the results reveal about whether the diagnosis was correct.

375 **Step 5 (Evidence Accumulation).** The model records the round’s outcome for future reference.

376 Record the outcome of this debugging round. Summarize what
377 was tried, what happened, and what was learned. Include:
378 hypothesis, fix applied, predicted vs actual outcomes, what
379 can be ruled out, and what hypotheses remain.

380 **Unstructured Condition.** The model receives the trace and failure signal but is not forced through
 381 the five-step process.

382 You are an expert debugger. An agent attempted a task and
 383 failed. You have the full execution trace and the failure
 384 signal. Analyze the trace, identify what went wrong,
 385 explain why, and generate a specific fix with predicted
 386 outcomes.

387 **Signal-Only Condition.** The model receives only the failure signal and the task description.

388 You previously attempted this task and failed. The verifier
 389 reported: [failure signal]. Try the task again: [task
 390 description].

391 A.3 Per-Round Solve Rates

392 Table 4 reports the cumulative solve rate at each round for all conditions on both benchmarks.

Table 4: Cumulative solve rate (%) at each round for all conditions and benchmarks.

Dataset	Condition	R1	R2	R3	R4	R5
SWE	Full	16.4	25.1	30.2	32.7	37.1
SWE	Unstructured	13.5	16.4	17.8	21.1	23.6
SWE	SignalOnly	15.3	17.8	19.3	21.5	21.5
TAU2	Full	26.2	48.0	59.6	66.5	70.9
TAU2	Unstructured	24.4	46.9	56.7	62.9	66.9
TAU2	SignalOnly	24.0	39.3	50.9	57.5	61.1

393 A.4 Per-Repository Breakdown (SWE-BENCH)

394 Table 5 reports the solve rate of the Full condition on SWE-BENCH broken down by repository. Repos-
 395 itories involving symbolic manipulation (sympy) and web frameworks (django) have the highest
 396 solve rates, while numerical and scientific computing repositories (scikit-learn, astropy)
 397 are not solved by any condition.

Table 5: Solve rate (%) of Full on SWE-BENCH by repository.

Repository	Tasks	Solve Rate
sympy/sympy	45	57.8
django/django	117	46.2
pylint-dev/pylint	5	40.0
sphinx-doc/sphinx	34	38.2
pydata/xarray	14	28.6
pytest-dev/pytest	9	11.1
matplotlib/matplotlib	20	10.0
scikit-learn/scikit-learn	16	0.0
astropy/astropy	12	0.0
psf/requests	3	0.0

398 B Notation

399 We provide a list of the notation used throughout the paper in Table 6.

Symbol	Meaning
$\mathcal{T}$	task description
$\tau = (q_1, \dots, q_T)$	execution trace (sequence of typed steps)
q_t	step t in the trace
$q_t^{\text{reasoning}}, q_t^{\text{action}}, q_t^{\text{observation}}$	reasoning, action, observation step
T	number of steps in the trace
M	LLM agent
$V(\tau) \in \{\text{pass}, \text{fail}\}$	verifier function
Q	complaint / failure signal
$\ell : \{1, \dots, T\} \rightarrow \{\text{correct}, \text{suspicious}, \text{wrong}\}$	step labeling function
h_k	hypothesis at round k
I_k	intervention (fix plan) at round k
E_k	accumulated evidence through round k
K	iteration budget

Table 6: Notation

References

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *NeurIPS*, 2017.
- [2] others Cheng. Contextual drag: How errors in the context affect llm reasoning. *arXiv preprint arXiv:2602.04288*, 2026.
- [3] Chroma Research. Context rot. 2025. <https://research.trychroma.com/context-rot>.
- [4] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. *ICLR*, 2024.
- [5] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *ICLR*, 2024.
- [6] Ryo Kamoi, Yusen Saxena, Nan Luo, Jihyung Kim, Prashanth Goyal, Greg Durrett, and Ani Nenkova. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *TMLR*, 2024.
- [7] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Weng, Ankur Kamath, and Pieter Abbeel. Training language models to self-correct via reinforcement learning. *ICLR (Oral)*, 2025.
- [8] Guannan Liang and Qianqian Tong. Llm-powered ai agent systems and their applications in industry. *arXiv preprint arXiv:2505.16120*, 2025.
- [9] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3-4):293–321, 1992.
- [10] others Long. The self-verification dilemma. *arXiv preprint arXiv:2602.03485*, 2026.
- [11] Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Dover: Intervention-driven auto debugging for llm multi-agent systems. *arXiv preprint arXiv:2512.06749*, 2025.
- [12] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *NeurIPS*, 2023.
- [13] Jiacheng Miao et al. Correctbench: Automatic testbed generation for the evaluation of language models at correcting reasoning. *NeurIPS*, 2025.

- [14] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [15] Nebius AI. Scaling data collection for training swe agents. 2025. <https://huggingface.co/datasets/nebius/SWE-agent-trajectories>.
- [16] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? *ICLR*, 2024.
- [17] OpenAI. Gpt-5.4 system card, 2026. <https://openai.com/research/gpt-5-4-system-card>.
- [18] Others. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*, 2025.
- [19] Others. Self-correction bench: Uncovering and addressing the self-correction blind spot in large language models. *arXiv preprint arXiv:2507.02778*, 2025.
- [20] Others. Decomposing llm self-correction. *arXiv preprint arXiv:2601.00828*, 2025.
- [21] Others. How many tries does it take? iterative self-repair in llms. *arXiv preprint arXiv:2604.10508*, 2025.
- [22] others. Sets: Leveraging self-verification and self-correction for revised task-plan execution. *TMLR*, 2025.
- [23] Others. Tau2-bench: A benchmark for tool-agent-user interaction. *arXiv preprint*, 2026. <https://huggingface.co/datasets/snorkelai/Tau2-Bench-Airline-With-Code-Agents>.
- [24] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.
- [25] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- [26] Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.
- [27] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [28] Gladys Tyen, Hassan Mansbach, Peter Chen, Tony Moran, and Daniel Kang. Llm cannot find reasoning errors, but can correct them given the error location. *Findings of ACL*, 2024.
- [29] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *NeurIPS*, 2023.
- [30] Siyu Yuan, Ziang Leng, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Agent-r: Training language model agents to reflect via iterative self-training. *arXiv preprint arXiv:2501.11425*, 2025.
- [31] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. *ICML*, 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state that we introduce a structured self-debugging method and present an empirical study comparing it against ablations. The experimental results in Section 4 directly support these claims.

2. Limitations

480 Question: Does the paper discuss the limitations of the work performed by the authors?

481 Answer: [Yes]

482 Justification: We discuss limitations in Section 5, including the requirement for textual execution

483 traces and a verifier with at least a binary pass/fail signal.

484 **3. Theory assumptions and proofs**

485 Question: For each theoretical result, does the paper provide the full set of assumptions and a

486 complete (and correct) proof?

487 Answer: [N/A]

488 Justification: The paper does not include theoretical results. The causal framework in Section 2 is

489 used for motivation, not as a formal theorem.

490 **4. Experimental result reproducibility**

491 Question: Does the paper fully disclose all the information needed to reproduce the main experi-

492 mental results of the paper to the extent that it affects the main claims and/or conclusions of the

493 paper (regardless of whether the code and data are provided or not)?

494 Answer: [Yes]

495 Justification: We describe the datasets, conditions, model, and evaluation metrics in Section 4.

496 Prompt templates and additional details are provided in Section A.

497 **5. Open access to data and code**

498 Question: Does the paper provide open access to the data and code, with sufficient instructions to

499 faithfully reproduce the main experimental results, as described in supplemental material?

500 Answer: [Yes]

501 Justification: Code is available at our anonymized repository. Both benchmarks (SWE-bench and

502 Tau2-Bench) are publicly available. We provide scripts to reproduce all experimental conditions.

503 **6. Experimental setting/details**

504 Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters,

505 how they were chosen, type of optimizer) necessary to understand the results?

506 Answer: [Yes]

507 Justification: We specify the number of tasks (275 per benchmark), the debugger model (GPT-5.4),

508 the budget ($K = 5$ rounds), and the three experimental conditions in Section 4. No training or

509 hyperparameter tuning is involved as our method operates at inference time.

510 **7. Experiment statistical significance**

511 Question: Does the paper report error bars suitably and correctly defined or other appropriate

512 information about the statistical significance of the experiments?

513 Answer: [No]

514 Justification: We report solve rates over 275 tasks per condition without error bars. Each task is

515 run once per condition (deterministic given the model). We report per-round cumulative rates and

516 per-repository breakdowns to characterize variability.

517 **8. Experiments compute resources**

518 Question: For each experiment, does the paper provide sufficient information on the computer

519 resources (type of compute workers, memory, time of execution) needed to reproduce the experi-

520 ments?

521 Answer: [Yes]

522 Justification: All experiments use GPT-5.4 via API calls. We report the number of tasks and

523 rounds. SWE-bench tasks are evaluated using the official Docker-based harness. Additional

524 compute details are in Section A.

525 **9. Code of ethics**

526 Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS

527 Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

528 Answer: [Yes]

529 Justification: The research uses publicly available benchmarks and API-accessible models. No

530 human subjects or private data are involved.

531 **10. Broader impacts**

532 Question: Does the paper discuss both potential positive societal impacts and negative societal

533 impacts of the work performed?

534 Answer: [\[Yes\]](#)

535 Justification: Structured self-debugging can improve the reliability of deployed LLM agents

536 (positive). We do not foresee specific negative societal impacts beyond those inherent to improving

537 LLM capabilities generally.

538 **11. Safeguards**

539 Question: Does the paper describe safeguards that have been put in place for responsible release

540 of data or models that have a high risk for misuse (e.g., pre-trained language models, image

541 generators, or scraped datasets)?

542 Answer: [\[N/A\]](#)

543 Justification: We do not release new models or datasets with high risk for misuse. Our contribution

544 is a prompting method evaluated on existing benchmarks.

545 **12. Licenses for existing assets**

546 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the

547 paper, properly credited and are the license and terms of use explicitly mentioned and properly

548 respected?

549 Answer: [\[Yes\]](#)

550 Justification: We cite SWE-bench and Tau2-Bench and their original papers. Both are publicly

551 available research benchmarks.

552 **13. New assets**

553 Question: Are new assets introduced in the paper well documented and is the documentation

554 provided alongside the assets?

555 Answer: [\[Yes\]](#)

556 Justification: We release code for our method and experimental pipeline at our anonymized

557 repository with documentation.

558 **14. Crowdsourcing and research with human subjects**

559 Question: For crowdsourcing experiments and research with human subjects, does the paper

560 include the full text of instructions given to participants and screenshots, if applicable, as well as

561 details about compensation (if any)?

562 Answer: [\[N/A\]](#)

563 Justification: The paper does not involve crowdsourcing or human subjects.

564 **15. Institutional review board (IRB) approvals or equivalent for research with human subjects**

565 Question: Does the paper describe potential risks incurred by study participants, whether such

566 risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals

567 (or an equivalent approval/review based on the requirements of your country or institution) were

568 obtained?

569 Answer: [\[N/A\]](#)

570 Justification: The paper does not involve human subjects research.

571 **16. Declaration of LLM usage**

572 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard

573 component of the core methods in this research?

574 Answer: [\[Yes\]](#)

575 Justification: LLMs are central to our method. We use GPT-5.4 as the self-debugging agent across

576 all conditions. The model is specified in Section 4.

577